

Q1_ArmstrongClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Q1_ArmstrongClient {
    public static void main(String[] args) throws Exception {
        // connect to the server running on same computer
        Socket s = new Socket("localhost", 5000);

        DataInputStream in = new DataInputStream(s.getInputStream());
        DataOutputStream out = new DataOutputStream(s.getOutputStream());

        // take number from user and send it to server
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = sc.nextInt();
        out.writeInt(num);

        // read the reply from server
        String reply = in.readUTF();
        System.out.println("Server says: " + reply);

        s.close();
    }
}
```

Q1_ArmstrongServer.java

```
import java.io.*;
import java.net.*;

public class Q1_ArmstrongServer {
    public static void main(String[] args) throws Exception {
        // create server on port 5000
        ServerSocket ss = new ServerSocket(5000);
        System.out.println("Server is waiting for client...");

        // wait until a client connects
        Socket s = ss.accept();
        System.out.println("Client connected");

        // streams to read from and write to the client
        DataInputStream in = new DataInputStream(s.getInputStream());
        DataOutputStream out = new DataOutputStream(s.getOutputStream());

        // read the number sent by client
        int num = in.readInt();
        System.out.println("Number received: " + num);

        // count the digits
        int temp = num;
        int digits = 0;
        while (temp > 0) {
            digits++;
            temp = temp / 10;
        }

        // add each digit raised to the power of number of digits
        temp = num;
        int sum = 0;
        while (temp > 0) {
            int digit = temp % 10;
            sum = sum + (int) Math.pow(digit, digits);
            temp = temp / 10;
        }

        // send the result back to client
        if (sum == num) {
            out.writeUTF(num + " is an Armstrong number");
        } else {
            out.writeUTF(num + " is not an Armstrong number");
        }

        s.close();
        ss.close();
    }
}
```

Q2_UDPClient.java

```
import java.net.*;
import java.util.Scanner;

public class Q2_UDPClient {
    public static void main(String[] args) throws Exception {
        DatagramSocket ds = new DatagramSocket();
        InetAddress ip = InetAddress.getByName("localhost");
        Scanner sc = new Scanner(System.in);

        while (true) {
            // type a message and send it to the server
            System.out.print("Enter message: ");
            String msg = sc.nextLine();
            byte[] data = msg.getBytes();
            DatagramPacket sendPacket = new DatagramPacket(data, data.length, ip, 5001);
            ds.send(sendPacket);

            // receive the reply from server
            byte[] buffer = new byte[1024];
            DatagramPacket receivePacket = new DatagramPacket(buffer, buffer.length);
            ds.receive(receivePacket);
            String reply = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Server: " + reply);
        }
    }
}
```

Q2_UDPServer.java

```
import java.net.*;
import java.util.Scanner;

public class Q2_UDPServer {
    public static void main(String[] args) throws Exception {
        // server socket on port 5001
        DatagramSocket ds = new DatagramSocket(5001);
        Scanner sc = new Scanner(System.in);
        System.out.println("UDP Server started, waiting for client...");

        while (true) {
            // receive message from client
            byte[] buffer = new byte[1024];
            DatagramPacket receivePacket = new DatagramPacket(buffer, buffer.length);
            ds.receive(receivePacket);
            String msg = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Client: " + msg);

            // type a reply and send it back to the client
            System.out.print("Enter reply: ");
            String reply = sc.nextLine();
            byte[] data = reply.getBytes();
            DatagramPacket sendPacket = new DatagramPacket(data, data.length,
                receivePacket.getAddress(), receivePacket.getPort());
            ds.send(sendPacket);
        }
    }
}
```

Q3_URLInfo.java

```
import java.net.URL;

public class Q3_URLInfo {
    public static void main(String[] args) throws Exception {
        // create a URL object
        URL u = new URL("https://www.example.com:8080/docs/index.html?id=5#top");

        // print information using URL methods
        System.out.println("Protocol : " + u.getProtocol());
        System.out.println("Host      : " + u.getHost());
        System.out.println("Port      : " + u.getPort());
        System.out.println("Path      : " + u.getPath());
        System.out.println("File      : " + u.getFile());
        System.out.println("Query     : " + u.getQuery());
        System.out.println("Ref       : " + u.getRef());
    }
}
```

Q4_GUIClient.java

```
import java.awt.*;
import java.awt.event.*;
import java.net.*;

public class Q4_GUIClient extends Frame implements ActionListener {
    TextArea chat = new TextArea();
    TextField msg = new TextField();
    Button send = new Button("Send");
    DatagramSocket ds;

    Q4_GUIClient() throws Exception {
        // client receives on port 5002 and sends to server on port 5001
        ds = new DatagramSocket(5002);

        // chat area in the middle, text box and button at the bottom
        setTitle("Client");
        setLayout(new BorderLayout());
        add(chat, BorderLayout.CENTER);
        Panel bottom = new Panel(new BorderLayout());
        bottom.add(msg, BorderLayout.CENTER);
        bottom.add(send, BorderLayout.EAST);
        add(bottom, BorderLayout.SOUTH);
        setSize(300, 300);
        setVisible(true);

        send.addActionListener(this);

        // close the window
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }

    // runs when Send button is clicked
    public void actionPerformed(ActionEvent e) {
        try {
            String text = msg.getText();
            byte[] data = text.getBytes();
            DatagramPacket p = new DatagramPacket(data, data.length,
                InetAddress.getByName("localhost"), 5001);
            ds.send(p);
            chat.append("Me: " + text + "\n");
            msg.setText("");
        } catch (Exception ex) {
            System.out.println(ex);
        }
    }

    public static void main(String[] args) throws Exception {
        Q4_GUIClient f = new Q4_GUIClient();

        // keep waiting for messages from the server
        while (true) {
            byte[] buffer = new byte[1024];
            DatagramPacket p = new DatagramPacket(buffer, buffer.length);
            f.ds.receive(p);
        }
    }
}
```

```
        String text = new String(p.getData(), 0, p.getLength());  
        f.chat.append("Server: " + text + "\n");  
    }  
}
```

Q4_GUIServer.java

```
import java.awt.*;
import java.awt.event.*;
import java.net.*;

public class Q4_GUIServer extends Frame implements ActionListener {
    TextArea chat = new TextArea();
    TextField msg = new TextField();
    Button send = new Button("Send");
    DatagramSocket ds;

    Q4_GUIServer() throws Exception {
        // server receives on port 5001 and sends to client on port 5002
        ds = new DatagramSocket(5001);

        // chat area in the middle, text box and button at the bottom
        setTitle("Server");
        setLayout(new BorderLayout());
        add(chat, BorderLayout.CENTER);
        Panel bottom = new Panel(new BorderLayout());
        bottom.add(msg, BorderLayout.CENTER);
        bottom.add(send, BorderLayout.EAST);
        add(bottom, BorderLayout.SOUTH);
        setSize(300, 300);
        setVisible(true);

        send.addActionListener(this);

        // close the window
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }

    // runs when Send button is clicked
    public void actionPerformed(ActionEvent e) {
        try {
            String text = msg.getText();
            byte[] data = text.getBytes();
            DatagramPacket p = new DatagramPacket(data, data.length,
                InetAddress.getByName("localhost"), 5002);
            ds.send(p);
            chat.append("Me: " + text + "\n");
            msg.setText("");
        } catch (Exception ex) {
            System.out.println(ex);
        }
    }

    public static void main(String[] args) throws Exception {
        Q4_GUIServer f = new Q4_GUIServer();

        // keep waiting for messages from the client
        while (true) {
            byte[] buffer = new byte[1024];
            DatagramPacket p = new DatagramPacket(buffer, buffer.length);
            f.ds.receive(p);
        }
    }
}
```

```
        String text = new String(p.getData(), 0, p.getLength());  
        f.chat.append("Client: " + text + "\n");  
    }  
}
```